Lab · From the registry to a container

MLOps · Master in Business Analytics and Data Science · MBDS-PT2026F · Session 5 (optional, self-paced)

About 30 minutes, plus 10–15 minutes the first time to install Docker. You type a few commands; you do not write code.

What this is about

In Session 3 you saw a **model registry**: the shortlist of models that matter, each with a version and a label. Version 1 of `churn-model` carries the label **production**. In Session 5 you saw that version packed into a **container** and answering customers live.

In this lab you do it yourself, on your own laptop:

1. Take the model exactly as the registry stores it.
2. Build the box: the model plus everything it needs to run.
3. Start the box: a small web service, waiting for requests.
4. Send it customers and read its answers.
5. Look inside, break it on purpose, and stop it.

At the end you will have done, in miniature, what "deploying a model" means in a company.

Before you start

You need:

- A laptop (Mac, Windows or Linux) where you can install software. **Company laptops often block this**: if yours does, skip to *"If you cannot install anything"* at the end.
- About 2 GB of free disk space and a normal internet connection (the first build downloads about 300 MB).
- The lab folder from the campus: **`MLOps_S05_Registry_to_Container_Lab.zip`**. Unzip it somewhere easy, such as your Desktop. You get a folder `churn_model_box` with:

File	What it is
`model/`	The model as the registry stores it: `MLmodel` (the label on the box: which inputs, which libraries, which run made it), `model.skops` (the trained model), `requirements.txt` (the exact library versions)
`Dockerfile`	The recipe for the box: eight lines
`customers.json`	Two customers to send to the model
`my_customer.json`	One customer for you to change

Install a container engine (once). Choose one; both are free:

- **Rancher Desktop** (open source, free for everyone, including on company machines): rancherdesktop.io. During setup, choose the container engine **dockerd (moby)**.
- **Docker Desktop**: docker.com. Free for personal use, education and small companies; large companies need a paid licence.

Open it and wait until it says it is **running** or **ready**. On Windows the installer may ask to enable "WSL 2"; accept and restart.

Open a terminal in the lab folder:

- **Mac:** right-click the `churn_model_box` folder in Finder → *New Terminal at Folder* (or open Terminal and type `cd` followed by dragging the folder into the window, then Enter).
- **Windows:** open the folder in File Explorer, click in the address bar, type `powershell` and press Enter.

Check that everything is ready by typing:

```
docker version
```

You should see a **Client** section and a **Server** section. If it says "command not found" or "cannot connect", the container engine is not running yet: open it, wait, and try again.

Step 1 · Look at what came out of the registry (3 minutes)

Open the file `model/MLmodel` with any text editor (TextEdit, Notepad). Find three things:

- **`signature`** → **`inputs`**: the model expects exactly three columns, `tenure_months`, `monthly_charges` and `support_tickets`. This is the model's *contract*: what it accepts.
- **`run_id`**: the training run that produced it. In the registry this links back to the settings, the data version and the metrics (Session 3).
- **`sklearn_version`** and `python_version`: the exact versions it was trained with.

Then open `model/requirements.txt`: the exact library versions the box must install. And open `Dockerfile`:

```
FROM python:3.10.12-slim                          # 1. a slim Python, the same version
the model was trained with
WORKDIR /opt/ml
COPY model/requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt # 2. the model's exact libraries
COPY model /opt/ml/model                          # 3. the model itself
CMD ["mlflow", "models", "serve", ...]            # 4. a web server that answers
prediction requests
```

Question to keep in mind: if you shipped only `model.skops`, which of these would be missing on the other side?

Step 2 · Build the box (3–6 minutes the first time)

```
docker build -t churn-model:production .
```

(Note the full stop at the end: it means "the recipe is in this folder".)

You will see a lot of text: Docker downloads the slim Python, installs the libraries, and copies the model. It ends with a line like `naming to docker.io/library/churn-model:production done`. Check that the box exists:

```
docker images churn-model
```

One line: `churn-model:production`, about 900 MB. **The box is built, but switched off.**

> *What just happened:* the recipe became an **image**, a frozen box with everything inside. You could send this image to any machine with Docker, a colleague's laptop or a bank's cloud, and it would run exactly the same.

Step 3 · Start the box (1 minute)

```
docker run -d -p 5002:8080 --name churn-prod churn-model:production
docker ps
```

`docker ps` shows one running container called `churn-prod`. In plain words: the box is now a small web service, listening on "door" **5002** of your laptop.

Wait about ten seconds, then ask it whether it is alive:

- **Mac / Linux:** `curl -s -o /dev/null -w "%{http_code}\n" http://localhost:5002/ping`
- **Windows (PowerShell):** `curl.exe -s -o NUL -w "%{http_code}" http://localhost:5002/ping`

Answer: **`200`**, the web's way of saying "I'm alive". (If you get `000`, it is still starting: wait five seconds and try again.) This is the same question a load balancer asks every few seconds to decide whether to send traffic to a copy of the model.

Step 4 · Send it two customers (3 minutes)

Look at `customers.json`: a **loyal** customer (60 months, €45 a month, no support tickets) and an **at-risk** one (3 months, €110 a month, 5 tickets). Send them:

- **Mac / Linux:** `curl -s -X POST http://localhost:5002/invocations -H "Content-Type: application/json" -d @customers.json`
- **Windows (PowerShell):** `curl.exe -s -X POST http://localhost:5002/invocations -H "Content-Type: application/json" -d "@customers.json"`

Answer:

```
{"predictions": [0, 1]}
```

0 = the loyal customer is predicted to stay. **1** = the at-risk customer is predicted to leave, so the retention team should call. This is a prediction from the production model, through a network request, from a box: what "a deployed model" means.

Step 5 · Your own customer (5 minutes)

Open `my_customer.json`: one customer with 24 months, €70 a month and 1 support ticket. Send it (same command, with `@my_customer.json`; on Windows `"@my_customer.json"`). Then change **one number at a time**, save, and send again. Try to answer:

- **Support tickets:** keep 24 months and €70. How many tickets turn this customer from 0 to 1?
- **Tenure:** keep €70 and 2 tickets. Below how many months does the customer become a 1?
- **Price:** keep 24 months and 1 ticket. Around which monthly charge does the answer flip?

You have just done, by hand, what a business user does with a deployed model: ask "what if?" and get an answer in milliseconds.

Step 6 · Break it on purpose (3 minutes)

A deployed model must refuse bad input instead of guessing. Try two mistakes in `my_customer.json` (keep a copy of the original first):

1. **A missing column:** delete `"support_tickets"` from `columns` and its value from `data`, so only two columns remain. Send it.
2. **A wrong type:** write `"two years"` (in quotes) instead of `24`. Send it.

Each time the box answers with an **error** that explains what is wrong: the input does not match the model's signature (the contract from Step 1). Nothing is predicted. This is Session 3's "check what comes in", done by the box itself.

Step 7 · Look inside, then stop it (3 minutes)

The box keeps a record of every request it answered:

```
docker logs churn-prod
```

You will see lines like `POST /invocations HTTP/1.1" 200 OK` (and `400` for the requests you broke on purpose). In a company, these logs go to the monitoring system (Session 7).

You can also look inside the running box:

```
docker exec churn-prod ls /opt/ml/model
```

The same files you unzipped, now inside the box. Finally, stop and remove it:

```
docker rm -f churn-prod
```

The image stays on your laptop: `docker run` starts it again in seconds. **Rollback** in a company is exactly this: stop the new version's box and start the previous version's box, which still exists and still runs exactly as it did.

To free the disk space when you are done with the lab: `docker rmi churn-model:production`.

To think about (optional)

Your professor may ask you to post on the forum; if not, these are good questions to check your understanding:

1. Which of the seven parts of the "ML artefact" (Session 5, slide 13) did you find in the folder, and which were missing?

2. The box answered in milliseconds on your laptop. What would have to be added before it could serve IberBank's 40 million transactions a month? (Think of the scaling slides: copies, a load balancer, autoscaling.)
3. Step 6 showed the box refusing bad input. Which other check would you want *before* the box goes live: on the code, on the data, or on the model (the test pyramid)?

If something goes wrong

Problem	What to do
`docker: command not found` / "Cannot connect to the Docker daemon"	Open Rancher Desktop or Docker Desktop and wait until it is running; then open a **new** terminal window.
The build stops with a network or certificate error	Company networks sometimes block downloads. Try from home or a phone hotspot; once built, the box needs no internet.
`port is already allocated`	Something else uses 5002: use `-p 5003:8080` in Step 3 and `5003` in the `curl` commands.
`Conflict. The container name "/churn-prod" is already in use`	You started it before: `docker rm -f churn-prod`, then Step 3 again.
`ping` gives `000`	The box is still starting; wait five seconds and try again.
Windows: `curl` prints a long PowerShell error	Use `curl.exe` (with `.exe`), and put `"@customers.json"` in quotes, as shown.

If you cannot install anything

Read the steps anyway: every command above shows the answer you would see. The key moments are: `docker images` shows the box (`churn-model:production`, about 900 MB); the ping answers `200`; the two customers return `{"predictions": [0, 1]}`; a missing column returns an error mentioning the model's schema; `docker logs` shows one line per request. The live demo in Session 5 showed the same steps.

Going further: `S05/deploy_lab` (on the campus) builds a box with a nicer interface (FastAPI, with an interactive page at `/docs`) and deploys it to Google Cloud Run.